Linux — Interview Ready Answers

Comprehensive answers with examples for all 16 Linux questions

16 Questions • Process Management • File System • Networking • Shell Scripting • LVM • Performance Tuning • System Administration

Q1. How do you check running processes in Linux?

Answer

Linux provides multiple tools for inspecting running processes, each suited to different use cases — from quick snapshots to real-time monitoring. Knowing all of them demonstrates solid systems knowledge.

1. ps — Process Snapshot

ps prints a snapshot of current processes at the moment it runs. It is the most fundamental process inspection tool.

```
# Show ALL processes in full detail (most useful)
ps aux
# a = all users, u = user-oriented format, x = include processes without TTY

# Output columns:
# USER  PID  %CPU  %MEM  VSZ    RSS   TTY  STAT  START  TIME  COMMAND
# root  1234  2.1   0.5  512MB  20MB  ?    Ss   10:00  0:02  nginx

# Show process tree (parent-child relationships)
ps axjf
ps aux --forest

# Find a specific process by name
ps aux | grep nginx
ps aux | grep -v grep | grep nginx   # Exclude the grep itself

# Sort by CPU usage (descending)
ps aux --sort=-%cpu | head -10

# Sort by memory usage
ps aux --sort=-%mem | head -10

# Show specific columns
ps -eo pid,ppid,cmd,%cpu,%mem --sort=-%cpu | head -20
# pid=Process ID, ppid=Parent PID, cmd=Command
```

ps STAT Column Values

STAT Code	Meaning
S	Sleeping — waiting for event (normal)
R	Running or runnable (in CPU queue)
D	Uninterruptible sleep — waiting for I/O (disk, network)
Z	Zombie — finished but parent hasn't read exit code
T	Stopped (by signal or job control)
s	Session leader
+	In foreground process group
<	High priority (nice < 0)
N	Low priority (nice > 0)

2. top — Real-Time Interactive Monitor

top provides a live, updating view of system resources and processes. It refreshes every 3 seconds by default and allows interactive sorting and process management.

```
top
# Interactive keys inside top:
# P = sort by CPU (default)
# M = sort by Memory
# k = kill a process (prompts for PID)
```

```
# r = renice a process
# 1 = show individual CPU cores
# q = quit
# < / > = move sort column left/right

# Non-interactive (one snapshot, useful for scripting)
top -bn1 | head -20
top -bn1 -p <PID>   # Monitor specific process

# top header interpretation:
# load average: 0.52, 0.41, 0.38  (1m, 5m, 15m averages)
# If load > number of CPU cores: system is overloaded
```

3. htop — Enhanced Interactive Monitor

htop is a more user-friendly, colourful version of top with mouse support, horizontal and vertical scrolling, and easier process management.

```
htop
# If not installed: apt install htop / yum install htop
# F5 = tree view, F6 = sort, F9 = kill, F10 = quit

# Non-interactive (for scripting)
htop --sort-key PERCENT_CPU -d 1 --no-color
```

4. pgrep and pstree

```
# Find PID by name
pgrep nginx
pgrep -l nginx      # -l shows process name too
pgrep -u appuser    # All processes by user
pgrep -a nginx      # Full command line

# Kill by name without finding PID first
pkill nginx         # Send SIGTERM
pkill -9 nginx      # Send SIGKILL (force)

# Show process tree
pstree              # All processes
pstree -p           # With PIDs
pstree <username>   # Processes for a user
```

5. /proc Filesystem — Direct Kernel Data

```
# Every process has a directory in /proc/<PID>/
ls /proc/1234/

# Read process command line
cat /proc/1234/cmdline | tr '\0' ' '

# Read process environment variables
cat /proc/1234/environ | tr '\0' '\n'

# Check open files
ls -la /proc/1234/fd/

# Memory maps
cat /proc/1234/maps
```

6. pidof and lsof

```
# Get PID of a running program
pidof nginx
pidof sshd

# List open files for a process
lsof -p <PID>
lsof -i :8080          # What's using port 8080?
lsof -u appuser        # All files opened by user
```

Quick Reference — Which Tool to Use

Scenario	Best Tool
Quick snapshot of all processes	ps aux
Real-time monitoring, kill/renice	top or htop

Find PID of a named process	pgrep -l nginx
Check what's using a port	lsof -i :8080 or ss -tlnp
Debug why a process is stuck in D state	ps aux | grep ' D ' then check I/O
Find CPU-hungry processes	top -bn1 | sort by CPU or ps aux --sort=-%cpu
See process parent-child tree	pstree -p or ps axjf

> *Tip: For scripting and automation, combine ps with awk for reliable output: ps -eo pid,pcpu,pmem,comm --sort=-pcpu | awk 'NR>1 && $2>50 {print $0}' — this prints only processes consuming more than 50% CPU, with no header and no grep artifacts.*

Q2. What command would you use to find files larger than 100MB?

Answer

The find command is the primary tool for locating files based on size, age, type, permissions, and many other criteria. Knowing its syntax and combining it with other commands is essential for disk management.

Basic find for Files Larger Than 100MB

```
# Find files larger than 100MB in the current directory and subdirectories
find . -type f -size +100M

# Find in a specific directory
find /var/log -type f -size +100M

# Find across the entire filesystem
find / -type f -size +100M 2>/dev/null
# 2>/dev/null suppresses 'Permission denied' errors
```

Size Units in find

Suffix	Unit
c	Bytes
k	Kilobytes (1024 bytes)
M	Megabytes
G	Gigabytes
+N	Greater than N
-N	Less than N
N (no sign)	Exactly N

Display with Human-Readable File Sizes

```
# Show size alongside each file
find / -type f -size +100M -exec ls -lh {} \; 2>/dev/null

# Cleaner: use printf for controlled output
find / -type f -size +100M -printf '%s\t%p\n' 2>/dev/null | \
  sort -rn | head -20
# %s = size in bytes, %p = full path

# Convert to MB in output
find / -type f -size +100M -printf '%.0fMB\t%p\n' 2>/dev/null | \
  awk '{printf "%.0f MB\t%s\n", $1/1048576, $2}' | sort -rn
```

Combining find with du and sort

```
# Find and show human-readable sizes, sorted largest first
find /var -type f -size +100M -exec du -sh {} + 2>/dev/null | \
  sort -rh
# -rh = reverse human-readable sort (1G, 500M, 100M...)

# Top 20 largest files across the system
find / -type f -printf '%s %p\n' 2>/dev/null | \
  sort -rn | head -20 | \
```

```
  awk '{printf "%.1f MB\t%s\n", $1/1048576, $2}'
```

Filtering by Additional Criteria

```
# Files larger than 100MB AND older than 30 days
find /var/log -type f -size +100M -mtime +30

# Files larger than 100MB modified in last 7 days
find /home -type f -size +100M -mtime -7

# Files larger than 100MB with .log extension
find /var/log -type f -size +100M -name '*.log'

# Files larger than 100MB owned by a specific user
find /home -type f -size +100M -user appuser
```

Taking Action on Found Files

```
# Delete all .log files larger than 100MB (use carefully!)
find /var/log -type f -size +100M -name '*.log' -delete

# Compress files larger than 100MB
find /var/log -type f -size +100M -exec gzip {} \;

# Move to archive directory
find /var/log -type f -size +100M \
  -exec mv {} /archive/logs/ \;

# Dry run first — just print what would be deleted
find /var/log -type f -size +100M -name '*.log' -print
```

Alternative Tools

```
# du — summarise directory sizes
du -sh /var/log/*   # Size of each item in /var/log
du -ah / | sort -rh | head -20   # Top 20 largest files/dirs

# ncdu — interactive disk usage viewer
ncdu /var
# Navigate with arrow keys, 'd' to delete

# df — filesystem disk usage
df -h              # Human-readable sizes
df -h /home        # Specific filesystem
```

Tip: Always do a dry run (omit -delete or -exec rm) before deleting. Run find with -print first to review the list of files that would be affected. A common mistake is running find with -delete at the wrong path and losing important data.

Q3. How do you check network connectivity between two servers?

Answer

Network connectivity troubleshooting requires a layered approach — testing from basic reachability (layer 3) through to application-level connectivity (layer 7). Each tool tests a different layer of the network stack.

Layer 3 — ICMP Reachability

```
# Basic ping — tests ICMP reachability
ping <hostname-or-IP>
ping -c 4 192.168.1.100   # Send exactly 4 packets
ping -i 0.5 server2       # Ping every 0.5 seconds
ping -s 1400 server2      # Test with large packet (MTU issues)
ping6 server2             # IPv6 ping

# Interpret results:
# Reply from 192.168.1.100: bytes=32 time=1ms TTL=64  → Connected
# Request timeout  → ICMP blocked or host unreachable
# Destination host unreachable  → No route to host
```

Layer 3 — Routing and Path

```
# traceroute — show every hop between source and destination
traceroute 192.168.1.100
traceroute -n 192.168.1.100   # -n skips DNS lookup (faster)
traceroute -p 80 192.168.1.100  # Use TCP port 80 instead of UDP
```

```
# mtr — combines ping and traceroute, continuously updates
mtr 192.168.1.100
mtr --report --report-cycles 10 192.168.1.100   # Non-interactive
# Shows per-hop latency, packet loss % — best for diagnosing drops
```

Layer 4 — TCP/UDP Port Connectivity

```
# nc (netcat) — test if a specific port is open
nc -zv 192.168.1.100 8080
# -z = scan without sending data, -v = verbose
# Output: Connection to 192.168.1.100 8080 port [tcp/*] succeeded!
# Output: nc: connect to 192.168.1.100 port 8080 (tcp) failed

# Test multiple ports
nc -zv 192.168.1.100 80 443 8080 8443

# UDP port test
nc -zvu 192.168.1.100 53   # -u = UDP

# Timeout after 3 seconds
nc -zvw 3 192.168.1.100 8080

# curl — test HTTP/HTTPS connectivity
curl -v http://192.168.1.100:8080/health
curl -I https://server2/api   # Headers only
curl --connect-timeout 5 http://server2:8080
```

Checking What Ports Are Listening

```
# ss — modern replacement for netstat
ss -tlnp   # TCP, listening, numeric, with process
ss -ulnp   # UDP, listening, numeric, with process
ss -tlnp | grep :8080   # Specific port

# netstat (older systems)
netstat -tlnp
netstat -tlnp | grep :80

# Check if port 443 is listening
ss -tlnp | grep ':443'
# tcp  LISTEN  0  128  0.0.0.0:443  0.0.0.0:*  users:(("nginx",pid=1234,fd=6))
```

DNS Resolution

```
# nslookup — basic DNS query
nslookup server2.company.com
nslookup server2.company.com 8.8.8.8   # Using specific DNS server

# dig — detailed DNS query
dig server2.company.com
dig server2.company.com +short    # Just the IP
dig server2.company.com MX        # MX records
dig @8.8.8.8 server2.company.com  # Query specific DNS server

# host — simple DNS lookup
host server2.company.com
```

Bandwidth and Throughput Testing

```
# iperf3 — measure network throughput between two servers
# On server: iperf3 -s
# On client:
iperf3 -c 192.168.1.100
iperf3 -c 192.168.1.100 -t 30   # 30-second test
iperf3 -c 192.168.1.100 -P 4    # 4 parallel streams
```

Network Interface and Route Information

```
# Check network interfaces
ip addr show
ip addr show eth0

# Check routing table
ip route show
ip route get 8.8.8.8   # Which route would be used to reach 8.8.8.8?
```

```
# Check ARP table (local network)
arp -n
ip neigh show
```

Firewall and Connection State

```
# Show active connections
ss -tnp state established
ss -tnp | grep ESTABLISHED

# Check iptables rules (if relevant)
iptables -L -n -v | grep -i 'DROP\|REJECT'

# Check if nftables is blocking
nft list ruleset | grep -i 'drop\|reject'
```

Systematic Troubleshooting Flow

Step	Tool
1. Basic reachability	ping server2
2. Route to host	ip route get <IP>
3. Hop-by-hop path	traceroute / mtr
4. Port reachability	nc -zv server2 8080
5. DNS resolution	dig server2.company.com
6. Application level	curl http://server2:8080
7. Firewall rules	iptables -L / nft list
8. Listening services	ss -tlnp on server2

> *Tip: Always test from both directions. A server may be reachable from A to B but not B to A if return traffic is blocked by a firewall. nc -zv tests TCP SYN — a success means the port is open and accepting connections, even if the firewall allows the SYN but blocks subsequent packets.*

Q4. Write a Python script to backup all files older than 30 days from a directory.

Answer

This coding question tests practical Python scripting ability — particularly the standard library modules for filesystem operations, date handling, and error management. Below is a complete, production-grade backup script.

Complete Python Backup Script

```
#!/usr/bin/env python3
"""
backup_old_files.py — Backup files older than N days from a source directory.
Usage: python3 backup_old_files.py [--source /path] [--dest /backup] [--days 30]
"""

import os
import shutil
import time
import logging
import argparse
import hashlib
from pathlib import Path
from datetime import datetime, timedelta


def setup_logging(log_file: str = '/var/log/backup.log') -> logging.Logger:
    """Configure logging to both console and file."""
    logger = logging.getLogger('backup')
    logger.setLevel(logging.INFO)
```

```
    formatter = logging.Formatter(
        '%(asctime)s [%(levelname)s] %(message)s',
        datefmt='%Y-%m-%d %H:%M:%S'
    )

    # Console handler
    ch = logging.StreamHandler()
    ch.setFormatter(formatter)
    logger.addHandler(ch)

    # File handler (with rotation)
    try:
        from logging.handlers import RotatingFileHandler
        fh = RotatingFileHandler(log_file, maxBytes=5*1024*1024, backupCount=5)
        fh.setFormatter(formatter)
        logger.addHandler(fh)
    except PermissionError:
        logger.warning(f'Cannot write to {log_file}, logging to console only')

    return logger


def compute_checksum(file_path: Path) -> str:
    """Compute MD5 checksum for file integrity verification."""
    md5 = hashlib.md5()
    with open(file_path, 'rb') as f:
        for chunk in iter(lambda: f.read(8192), b''):
            md5.update(chunk)
    return md5.hexdigest()


def backup_old_files(
    source_dir: str,
    dest_dir: str,
    days_old: int = 30,
    dry_run: bool = False,
    logger: logging.Logger = None
) -> dict:
    """
    Backup files older than days_old from source_dir to dest_dir.

    Returns: dict with backed_up, skipped, errors counts
    """
    if logger is None:
        logger = logging.getLogger('backup')

    source = Path(source_dir)
    dest   = Path(dest_dir)
    cutoff = datetime.now() - timedelta(days=days_old)
    cutoff_ts = cutoff.timestamp()

    # Validate source directory exists
    if not source.exists():
        raise FileNotFoundError(f'Source directory not found: {source}')
    if not source.is_dir():
        raise NotADirectoryError(f'Source is not a directory: {source}')

    # Create destination directory if it doesn't exist
    if not dry_run:
        dest.mkdir(parents=True, exist_ok=True)
        logger.info(f'Backup destination: {dest}')

    stats = {'backed_up': 0, 'skipped': 0, 'errors': 0,
             'total_bytes': 0, 'files': []}

    logger.info(f'Scanning {source} for files older than {days_old} days...')
    logger.info(f'Cutoff date: {cutoff.strftime("%Y-%m-%d %H:%M:%S")}')

    # Walk all files recursively
    for file_path in source.rglob('*'):
```

```
        if not file_path.is_file():
            continue

        try:
            stat = file_path.stat()
            file_mtime = stat.st_mtime

            # Skip files modified within the cutoff period
            if file_mtime >= cutoff_ts:
                stats['skipped'] += 1
                continue

            # Preserve directory structure in backup
            relative_path = file_path.relative_to(source)
            backup_path   = dest / relative_path

            file_age_days = (time.time() - file_mtime) / 86400
            file_size     = stat.st_size

            if dry_run:
                logger.info(f'[DRY-RUN] Would backup: {relative_path} '
                            f'({file_size/1024:.1f}KB, {file_age_days:.0f} days old)')
                stats['backed_up'] += 1
                stats['total_bytes'] += file_size
                continue

            # Create parent directories
            backup_path.parent.mkdir(parents=True, exist_ok=True)

            # Compute checksum before copy
            src_checksum = compute_checksum(file_path)

            # Copy file preserving metadata (timestamps, permissions)
            shutil.copy2(str(file_path), str(backup_path))

            # Verify integrity after copy
            dest_checksum = compute_checksum(backup_path)
            if src_checksum != dest_checksum:
                raise IOError(f'Checksum mismatch after copy: {file_path}')

            logger.info(f'Backed up: {relative_path} '
                        f'({file_size/1024:.1f}KB, {file_age_days:.0f} days old)')

            stats['backed_up']   += 1
            stats['total_bytes'] += file_size
            stats['files'].append(str(relative_path))

        except PermissionError as e:
            logger.error(f'Permission denied: {file_path} — {e}')
            stats['errors'] += 1
        except OSError as e:
            logger.error(f'OS error with {file_path}: {e}')
            stats['errors'] += 1
        except Exception as e:
            logger.error(f'Unexpected error with {file_path}: {e}')
            stats['errors'] += 1

    return stats


def send_notification(stats: dict, source: str, dry_run: bool):
    """Send email notification (or write to webhook) about backup results."""
    import smtplib
    from email.mime.text import MIMEText

    subject = f'Backup {'(DRY-RUN) ' if dry_run else ''}Complete: {source}'
    body = (
        f'Backup Summary\n'
        f'  Source:     {source}\n'
```

```
        f'  Backed up:  {stats["backed_up"]} files'
        f' ({stats["total_bytes"]/1024/1024:.1f} MB)\n'
        f'  Skipped:    {stats["skipped"]} files (too new)\n'
        f'  Errors:     {stats["errors"]}\n'
    )

    # Send email — configure SMTP settings
    try:
        msg = MIMEText(body)
        msg['Subject'] = subject
        msg['From']    = 'backup@company.com'
        msg['To']      = 'ops@company.com'
        with smtplib.SMTP('localhost') as smtp:
            smtp.send_message(msg)
    except Exception as e:
        print(f'Email notification failed: {e}')


def main():
    parser = argparse.ArgumentParser(description='Backup files older than N days')
    parser.add_argument('--source', default='/var/app/data',
                        help='Source directory to back up')
    parser.add_argument('--dest',   default='/backup/data',
                        help='Backup destination directory')
    parser.add_argument('--days',   type=int, default=30,
                        help='Files older than this many days (default: 30)')
    parser.add_argument('--dry-run', action='store_true',
                        help='Preview what would be backed up without copying')
    parser.add_argument('--notify', action='store_true',
                        help='Send email notification after backup')
    args = parser.parse_args()

    logger = setup_logging()
    logger.info('=' * 60)
    logger.info(f'Starting backup: {args.source} → {args.dest}')

    try:
        stats = backup_old_files(
            source_dir = args.source,
            dest_dir   = args.dest,
            days_old   = args.days,
            dry_run    = args.dry_run,
            logger     = logger
        )

        logger.info('=' * 60)
        logger.info(f'Backup complete.')
        logger.info(f'  Files backed up : {stats["backed_up"]}')
        logger.info(f'  Files skipped   : {stats["skipped"]} (newer than {args.days}
days)')
        logger.info(f'  Total size      : {stats["total_bytes"]/1024/1024:.2f} MB')
        logger.info(f'  Errors          : {stats["errors"]}')

        if args.notify:
            send_notification(stats, args.source, args.dry_run)

        # Exit with non-zero if there were errors
        exit(1 if stats['errors'] > 0 else 0)

    except (FileNotFoundError, NotADirectoryError) as e:
        logger.critical(f'Fatal error: {e}')
        exit(2)


if __name__ == '__main__':
    main()
```

Running the Script

```
# Dry run first — see what would be backed up
python3 backup_old_files.py \
  --source /var/app/data \
```

```
  --dest /backup/data \
  --days 30 \
  --dry-run

# Real backup with notification
python3 backup_old_files.py \
  --source /var/app/data \
  --dest /mnt/nas/backup \
  --days 30 \
  --notify

# Schedule with cron (daily at 2 AM)
0 2 * * * /usr/bin/python3 /opt/scripts/backup_old_files.py \
  --source /var/app/data --dest /backup/data --days 30 --notify
```

Key Features Explained

- Path.rglob('*') — Recursively finds all files in subdirectories without additional loops.
- shutil.copy2() — Copies files AND preserves metadata (modification time, access time, permissions). Use shutil.copy() if you don't want to preserve timestamps.
- Checksum verification — MD5 computed before and after copy to detect silent data corruption during transfer.
- Dry-run mode — Safe preview before actual backup. Always run this first in production.
- Structured error handling — Permission errors, OS errors, and unexpected errors are caught separately and counted without crashing the script.

Tip: For large-scale backups, replace shutil.copy2() with rsync via subprocess — rsync handles partial transfers, bandwidth throttling, and is significantly faster for large files: subprocess.run(['rsync', '-a', '--checksum', str(file_path), str(backup_path)])

Q5. Write a shell script to automate log rotation and system cleanup with error handling and notifications.

Answer

This comprehensive shell script demonstrates production-grade scripting practices: set -euo pipefail for strict error handling, structured logging, disk space awareness, and multi-channel notification support.

Complete Shell Script — Log Rotation and System Cleanup

```
#!/bin/bash
# ================================================================
# system_cleanup.sh — Log rotation, cleanup, and notifications
# Usage: sudo ./system_cleanup.sh [--dry-run] [--notify-slack]
# Requires: gzip, find, df, mail or curl (for notifications)
# ================================================================
set -euo pipefail
# -e = exit on error, -u = error on unset var, -o pipefail = pipe errors

# ── Configuration ──────────────────────────────────────────
readonly SCRIPT_NAME="$(basename "$0")"
readonly SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
readonly TIMESTAMP="$(date +%Y%m%d_%H%M%S)"
readonly LOG_FILE="/var/log/cleanup/${TIMESTAMP}_cleanup.log"
readonly LOCK_FILE="/tmp/system_cleanup.lock"

# Directories to rotate/cleanup
readonly APP_LOG_DIR="/var/log/myapp"
readonly SYSTEM_LOG_DIR="/var/log"
readonly TMP_DIR="/tmp"
readonly BACKUP_DIR="/var/log/archive"

# Thresholds
readonly DISK_WARNING_PCT=80      # Warn if disk > 80% full
readonly DISK_CRITICAL_PCT=90     # Alert if disk > 90% full
readonly LOG_MAX_AGE_DAYS=30      # Compress logs older than 30 days
readonly LOG_DELETE_AGE_DAYS=90   # Delete logs older than 90 days
readonly TMP_MAX_AGE_DAYS=7       # Delete temp files older than 7 days

# Notification settings
readonly SLACK_WEBHOOK="${SLACK_WEBHOOK:-}"
```

```
readonly ALERT_EMAIL="ops@company.com"

# ── Parse Arguments ──────────────────────────────────────
DRY_RUN=false
NOTIFY_SLACK=false
for arg in "$@"; do
    case "$arg" in
        --dry-run)     DRY_RUN=true ;;
        --notify-slack) NOTIFY_SLACK=true ;;
        --help)
            echo "Usage: $SCRIPT_NAME [--dry-run] [--notify-slack]"
            exit 0 ;;
        *) echo "Unknown argument: $arg"; exit 1 ;;
    esac
done

# ── Logging Functions ────────────────────────────────────
mkdir -p "$(dirname "$LOG_FILE")"

log() {
    local level="$1"; shift
    local message="$*"
    local timestamp
    timestamp="$(date '+%Y-%m-%d %H:%M:%S')"
    echo "${timestamp} [${level}] ${message}" | tee -a "$LOG_FILE"
}

info()     { log INFO    "$@"; }
warn()     { log WARNING "$@"; }
error()    { log ERROR   "$@" >&2; }
critical() { log CRITICAL "$@" >&2; }

# ── Lock File (Prevent Concurrent Runs) ──────────────────
if [ -f "$LOCK_FILE" ]; then
    OLD_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo 0)
    if kill -0 "$OLD_PID" 2>/dev/null; then
        critical "Another cleanup is running (PID=$OLD_PID). Exiting."
        exit 1
    else
        warn "Stale lock file found. Removing."
        rm -f "$LOCK_FILE"
    fi
fi
echo $$ > "$LOCK_FILE"

# Remove lock file on exit (including errors)
cleanup_lock() { rm -f "$LOCK_FILE"; }
trap cleanup_lock EXIT

# ── Counters ─────────────────────────────────────────────
FILES_COMPRESSED=0
FILES_DELETED=0
BYTES_FREED=0
ERRORS=0

# ── Helper: Get Disk Usage ───────────────────────────────
get_disk_usage() {
    local path="${1:-/}"
    df -h "$path" | awk 'NR==2 {print $5}' | tr -d '%'
}

get_disk_free() {
    local path="${1:-/}"
    df -h "$path" | awk 'NR==2 {print $4}'
}

# ── Helper: Execute or Dry-Run ───────────────────────────
execute() {
    if [ "$DRY_RUN" = true ]; then
        info "[DRY-RUN] Would run: $*"
```

```
    else
        eval "$@"
    fi
}

# ── Step 1: Check Disk Space ──────────────────────────────
check_disk_space() {
    info '━━━ Disk Space Check ━━━'
    local usage free
    usage=$(get_disk_usage '/')
    free=$(get_disk_free '/')
    info "Root filesystem: ${usage}% used, ${free} free"

    if [ "$usage" -ge "$DISK_CRITICAL_PCT" ]; then
        critical "DISK CRITICAL: ${usage}% used on /"
        send_notification "CRITICAL" "Disk usage at ${usage}% — immediate action required"
    elif [ "$usage" -ge "$DISK_WARNING_PCT" ]; then
        warn "DISK WARNING: ${usage}% used on /"
        send_notification "WARNING" "Disk usage at ${usage}%"
    fi
}

# ── Step 2: Rotate Application Logs ──────────────────────
rotate_app_logs() {
    info '━━━ Rotating Application Logs ━━━'

    if [ ! -d "$APP_LOG_DIR" ]; then
        warn "App log directory not found: $APP_LOG_DIR"
        return 0
    fi

    mkdir -p "$BACKUP_DIR"

    # Compress logs older than LOG_MAX_AGE_DAYS days
    while IFS= read -r -d '' file; do
        if [ "$DRY_RUN" = true ]; then
            info "[DRY-RUN] Would compress: $file"
            FILES_COMPRESSED=$((FILES_COMPRESSED + 1))
        else
            size=$(stat -c%s "$file" 2>/dev/null || echo 0)
            if gzip -9 "$file"; then
                FILES_COMPRESSED=$((FILES_COMPRESSED + 1))
                BYTES_FREED=$((BYTES_FREED + size * 65 / 100))  # ~65% reduction
                info "Compressed: $(basename "$file")"
            else
                error "Failed to compress: $file"
                ERRORS=$((ERRORS + 1))
            fi
        fi
    done < <(find "$APP_LOG_DIR" -type f -name '*.log' \
                 ! -name '*.gz' -mtime +"$LOG_MAX_AGE_DAYS" -print0)

    # Delete compressed logs older than LOG_DELETE_AGE_DAYS
    while IFS= read -r -d '' file; do
        if [ "$DRY_RUN" = true ]; then
            info "[DRY-RUN] Would delete: $file"
            FILES_DELETED=$((FILES_DELETED + 1))
        else
            size=$(stat -c%s "$file" 2>/dev/null || echo 0)
            if rm -f "$file"; then
                FILES_DELETED=$((FILES_DELETED + 1))
                BYTES_FREED=$((BYTES_FREED + size))
                info "Deleted: $(basename "$file")"
            else
                error "Failed to delete: $file"
                ERRORS=$((ERRORS + 1))
            fi
        fi
    done < <(find "$APP_LOG_DIR" -type f -name '*.gz' \
                  -mtime +"$LOG_DELETE_AGE_DAYS" -print0)
}
```

```
# ── Step 3: Clean Temp Files ──────────────────────────────
clean_temp_files() {
    info '━━━ Cleaning Temp Files ━━━'

    local count
    count=$(find "$TMP_DIR" -type f -atime +"$TMP_MAX_AGE_DAYS" 2>/dev/null | wc -l)
    info "Found $count temp files older than $TMP_MAX_AGE_DAYS days"

    if [ "$DRY_RUN" = true ]; then
        info "[DRY-RUN] Would delete $count temp files"
    else
        find "$TMP_DIR" -type f -atime +"$TMP_MAX_AGE_DAYS" \
             -not -name '*.lock' \
             -delete 2>/dev/null || true
        FILES_DELETED=$((FILES_DELETED + count))
        info "Deleted $count temp files"
    fi
}

# ── Step 4: Clean Package Cache ───────────────────────────
clean_package_cache() {
    info '━━━ Cleaning Package Cache ━━━'

    if command -v apt-get &>/dev/null; then
        execute 'apt-get clean -y'
        execute 'apt-get autoremove -y --purge'
        info 'APT cache cleaned'
    elif command -v yum &>/dev/null; then
        execute 'yum clean all'
        info 'YUM cache cleaned'
    elif command -v dnf &>/dev/null; then
        execute 'dnf clean all'
        info 'DNF cache cleaned'
    fi
}

# ── Step 5: Clean Docker Resources ────────────────────────
clean_docker() {
    if ! command -v docker &>/dev/null; then
        return 0
    fi
    info '━━━ Cleaning Docker Resources ━━━'
    execute 'docker system prune -f --volumes'
    info 'Docker resources cleaned'
}

# ── Notification ──────────────────────────────────────────
send_notification() {
    local level="${1:-INFO}"
    local message="$2"

    # Email notification
    if command -v mail &>/dev/null; then
        echo "$message" | mail -s "[${level}] System Cleanup: $HOSTNAME" "$ALERT_EMAIL" ||
true
    fi

    # Slack notification
    if [ "$NOTIFY_SLACK" = true ] && [ -n "$SLACK_WEBHOOK" ]; then
        local colour
        case "$level" in
            CRITICAL) colour='danger'  ;;
            WARNING)  colour='warning' ;;
            *)        colour='good'    ;;
        esac
        curl -s -X POST "$SLACK_WEBHOOK" \
          -H 'Content-Type: application/json' \
          -d "{\"attachments\":[{\"color\":\"${colour}\",
               \"title\":\"System Cleanup [${level}]\",
```

```
                \"text\":\"${message}\",
                \"footer\":\"${HOSTNAME} — $(date)\"}]}" >/dev/null || true
    fi
}

# ── Generate Summary ─────────────────────────────────────
generate_summary() {
    local freed_mb=$((BYTES_FREED / 1024 / 1024))
    local disk_after
    disk_after=$(get_disk_usage '/')

    info '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━'
    info 'CLEANUP SUMMARY'
    info '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━'
    info "  Files compressed : $FILES_COMPRESSED"
    info "  Files deleted    : $FILES_DELETED"
    info "  Space freed      : ~${freed_mb}MB"
    info "  Disk usage after : ${disk_after}%"
    info "  Errors           : $ERRORS"
    info "  Log file         : $LOG_FILE"
    info '━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━'

    local msg
    msg="Cleanup complete on $HOSTNAME. Freed ~${freed_mb}MB. Disk: ${disk_after}%."
    msg+="Errors: $ERRORS."
    send_notification 'INFO' "$msg"
}

# ── Main ─────────────────────────────────────────────────
main() {
    info '═══════════════════════════════════════════════'
    info "Starting system cleanup on $HOSTNAME"
    info "Timestamp: $(date)"
    [ "$DRY_RUN" = true ] && info 'DRY-RUN MODE — no changes will be made'
    info '═══════════════════════════════════════════════'

    check_disk_space
    rotate_app_logs
    clean_temp_files
    clean_package_cache
    clean_docker
    generate_summary

    [ "$ERRORS" -gt 0 ] && exit 1 || exit 0
}

main
```

Making It Executable and Scheduling

```
chmod +x system_cleanup.sh

# Test with dry-run first
sudo ./system_cleanup.sh --dry-run

# Run with Slack notification
sudo SLACK_WEBHOOK='https://hooks.slack.com/...' ./system_cleanup.sh --notify-slack

# Schedule with cron — every Sunday at 3 AM
0 3 * * 0 root /opt/scripts/system_cleanup.sh --notify-slack >> /var/log/cleanup.log 2>&1
```

Key Script Features

Feature	Implementation
Strict error mode	set -euo pipefail
Lock file	Check and create /tmp/*.lock
Dry-run mode	--dry-run flag
Null-delimited find	find -print0 with read -d ''

Trap on EXIT	trap cleanup_lock EXIT
Multi-channel alerts	Email + Slack
Timestamped logs	tee -a to log file

> *Tip: Always test cleanup scripts with --dry-run in a staging environment before scheduling in production. A bug in a cleanup script can cause irreversible data loss. Use find with -print before -delete to verify the file list is exactly what you intend to remove.*

Q6. Your Linux server is experiencing high load. How do you identify and resolve performance bottlenecks?

Answer

High server load requires systematic investigation across four resource domains: CPU, memory, disk I/O, and network. The goal is to move from symptoms (high load average) to root cause (specific process or resource) to resolution.

Step 1 — Establish the Symptom: Load Average

```
# Load average shows CPU demand over 1, 5, 15 minutes
uptime
# 10:30:00 up 5 days,  2:20,  1 user,  load average: 8.52, 6.41, 4.33

# Interpretation:
# Load = number of CPUs: system is at 100% capacity
# Load > number of CPUs: system is OVERLOADED (queue forming)

# Check how many CPUs/cores available
nproc
grep -c processor /proc/cpuinfo
```

A load average of 8.52 on an 8-core server means the system is at full capacity. On a 4-core server, it means 2x overloaded — tasks are waiting in queue.

Step 2 — CPU Analysis

```
# top: identify CPU-hungry processes
top -bn1 | sort -k9 -rn | head -15
# Column 9 is %CPU — sorted descending

# Check overall CPU breakdown
top -bn1 | grep 'Cpu(s)'
# Cpu(s): 45.2 us, 12.3 sy, 0.0 ni, 30.1 id, 10.5 wa, 0.2 hi, 1.7 si
# us=user, sy=system, id=idle, wa=iowait, hi=hardware interrupt, si=software interrupt

# High 'wa' (iowait > 10%) indicates disk I/O bottleneck
# High 'sy' (system > 30%) indicates kernel overhead
# High 'us' (user > 80%) indicates CPU-bound application

# mpstat — per-core CPU breakdown
mpstat -P ALL 1 5

# pidstat — per-process CPU usage over time
pidstat -u 1 10   # Every 1 second, 10 iterations
pidstat -p <PID> 1
```

Step 3 — Memory Analysis

```
# Overview of memory
free -h
# Mem:  16G  14G  512M  1.2G  1.5G  2.1G
# total used free shared buff/cache available

# Detailed memory info
cat /proc/meminfo

# Check swap usage (heavy swap use = memory pressure)
swapon --show
vmstat 1 5   # Virtual memory stats — check si/so (swap in/out)

# Processes consuming most memory
```

```
ps aux --sort=-%mem | head -10

# Check if OOM killer is active
dmesg | grep -i 'oom\|killed'
journalctl -k | grep -i 'oom\|out of memory'
```

Step 4 — Disk I/O Analysis

```
# iostat — disk utilisation and throughput
iostat -x 1 5
# Key columns: %util (device saturation), await (ms wait time)
# %util > 80%: disk is bottleneck
# await > 20ms: high I/O latency

# iotop — per-process I/O (like top but for disk)
iotop -o      # -o shows only processes with active I/O
iotop -b -n 5 # Non-interactive, 5 iterations

# Check for processes in 'D' (uninterruptible) state
ps aux | awk '$8 ~ /^D/ {print $0}'
# D state = waiting for I/O — too many = disk bottleneck

# Find what's filling disk
du -sh /* 2>/dev/null | sort -rh | head -10
lsof | grep deleted | awk '{print $7, $9}' | sort -rn | head -10
# Deleted files still open by processes take up disk space
```

Step 5 — Network Analysis

```
# nethogs — per-process network usage (like iotop for network)
nethogs eth0

# iftop — network traffic by connection
iftop -i eth0

# ss — connection states
ss -s   # Summary of connection states
ss -tnp | grep ESTABLISHED | wc -l   # Count active connections
ss -tnp | grep TIME_WAIT  | wc -l    # Time-wait connections

# Check for network saturation
cat /proc/net/dev
ip -s link show eth0   # Packet errors and drops
```

Step 6 — Specific Process Deep-Dive

```
# Once you've identified the problematic PID
strace -p <PID> -e trace=open,read,write,connect
# Shows system calls in real-time — reveals what the process is doing

# Check open file descriptors
lsof -p <PID>
ls /proc/<PID>/fd | wc -l   # Count open files
cat /proc/sys/fs/file-max   # System-wide file limit

# Check process threads
ls /proc/<PID>/task | wc -l

# CPU profiling (if perf available)
perf top -p <PID>
```

Common Root Causes and Resolutions

Symptom	Root Cause
High CPU us (user space)	CPU-bound application code
High CPU wa (iowait)	Slow disk I/O
High CPU sy (system)	Kernel overhead, too many threads/processes
Low memory, high swap	Memory leak or undersized instance
OOM killer triggered	Application using too much

	memory
Many D-state processes	Disk I/O hang or NFS mount issue
Load high but CPU idle	I/O wait or blocked processes
Network saturation	High traffic or broadcast storm

Resolution Actions

```
# Kill a runaway process
kill -15 <PID>   # SIGTERM — graceful shutdown request
kill -9  <PID>   # SIGKILL — force kill (last resort)

# Reduce process priority (nice value)
nice -n 19 <command>       # Start with lowest priority
renice 10 -p <PID>        # Reduce priority of running process

# Clear page cache (recovers memory temporarily)
sync && echo 3 > /proc/sys/vm/drop_caches

# Limit process resources with cgroups (Linux 2.6+)
systemctl set-property myapp.service CPUQuota=50%
systemctl set-property myapp.service MemoryLimit=2G
```

> *Tip: Use the USE method (Utilisation, Saturation, Errors) for systematic investigation: for each resource (CPU, memory, disk, network), check utilisation (how busy?), saturation (how much waiting?), and errors (any failures?). This prevents jumping to conclusions before gathering all data.*

Q7. What is LVM and why would you use it?

Answer

The existing answer is correct and complete. Expanding with full context, real-world scenarios, and operational details.

What is LVM

LVM (Logical Volume Manager) is an abstraction layer between physical storage devices and the filesystem. Instead of partitions being fixed in size and tied to specific physical disks, LVM creates a pool of storage that can be divided, resized, and managed dynamically without downtime.

The Three-Layer Architecture

```
Physical Disks: /dev/sdb, /dev/sdc, /dev/sdd
         ↓
PV  (Physical Volumes): /dev/sdb, /dev/sdc   ← add disks to LVM
         ↓
VG  (Volume Group):     vg_data              ← pool of storage
         ↓
LV  (Logical Volumes):  lv_app, lv_db, lv_logs  ← logical partitions
         ↓
Filesystem: ext4, xfs, etc. on each LV
```

Why Use LVM

- Online resizing — Extend logical volumes and filesystems while they are mounted and in use. No downtime required.
- Disk aggregation — Combine multiple physical disks into one large logical volume. A 1TB LV can span two 500GB physical disks seamlessly.
- Snapshots — Create point-in-time snapshots for backups or pre-upgrade safety nets, without stopping the application.
- Thin provisioning — Allocate more logical space than physically exists (overcommit), allocating physical space only as it's actually used.
- Disk migration — Move data from one physical disk to another (pvmove) without unmounting or stopping services.
- Flexibility — Add new disks at any time and extend the volume group and logical volumes dynamically.

LVM vs Traditional Partitioning

Feature	Traditional Partitions
Resize without unmounting	Not possible
Span multiple disks	Not possible
Snapshots	Not possible

Add storage online	Not possible
Thin provisioning	Not possible
Migration between disks	Requires downtime

Real-World Scenarios

- Application log volume fills up — extend lv_logs without restarting the application.
- Database needs more space — add a new disk, extend vg_data, extend lv_db, grow the filesystem — all while PostgreSQL is running.
- Pre-upgrade snapshot — snapshot lv_app before a major deployment; if it fails, revert the snapshot in seconds.
- Datacenter migration — pvmove data from old SATA disks to new NVMe SSDs without any service interruption.

> *Tip: LVM is a standard feature included in most Linux distributions and enabled by default in many enterprise Linux installations (RHEL, CentOS, Ubuntu Server). It adds no performance overhead for linear volumes and is transparent to the filesystem and applications above it.*

Q8. What are the main components of LVM?

Answer

The existing answer is correct. Expanding with complete details, commands, and visual representation.

1. PV — Physical Volume

A PV is a physical disk, partition, or device file that has been initialised for use by LVM. The pvcreate command writes LVM metadata to the beginning of the disk, making it available to be added to a Volume Group.

```
# View all physical volumes
pvs                   # Concise listing
pvdisplay             # Detailed info
pvdisplay /dev/sdb    # Specific PV

# pvs output columns:
# PV         VG       Fmt  Attr PSize  PFree
# /dev/sdb   vg_data  lvm2 a--  100GB  20GB
# /dev/sdc   vg_data  lvm2 a--  200GB  0

# Scan for all PVs in the system
pvs --all
pvscan
```

2. VG — Volume Group

A VG is a named pool of storage created from one or more Physical Volumes. All PVs in a VG share a common storage space that Logical Volumes are carved from. The VG is the abstraction layer that hides the physical disk boundaries from the logical volumes.

```
# View all volume groups
vgs                   # Concise
vgdisplay             # Detailed
vgdisplay vg_data     # Specific VG

# vgs output:
# VG      #PV #LV #SN Attr  VSize  VFree
# vg_data   2   3   1 wz--n  300GB  20GB
# 2 PVs, 3 LVs, 1 Snapshot, 300GB total, 20GB free
```

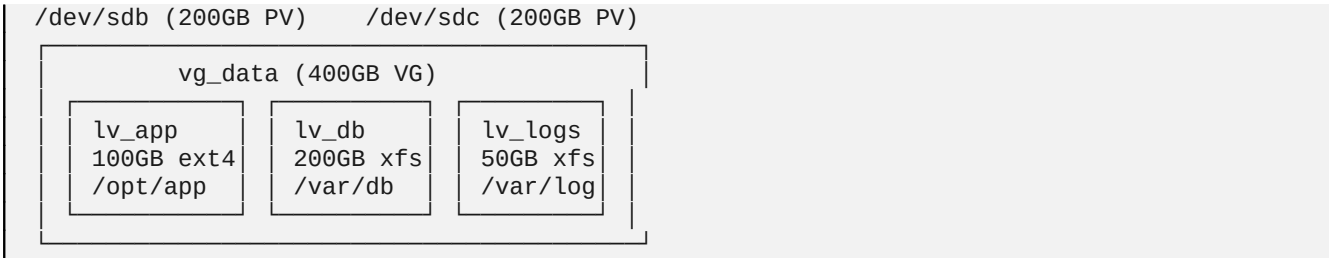
3. LV — Logical Volume

An LV is a logical partition created within a VG. It behaves exactly like a regular disk partition — you format it with a filesystem and mount it. Unlike partitions, LVs can be resized, snapshotted, and span multiple physical disks.

```
# View all logical volumes
lvs                   # Concise
lvdisplay             # Detailed
lvdisplay /dev/vg_data/lv_app  # Specific LV

# lvs output:
# LV      VG       Attr       LSize  Pool Origin Snap%
# lv_app  vg_data  -wi-ao---- 50GB
# lv_db   vg_data  -wi-ao---- 100GB
# lv_snap vg_data  swi-a-s---  5GB  lv_app       12.5%
```

Component Relationships Diagram



PE — Physical Extents

Internally, LVM divides Physical Volumes into fixed-size chunks called Physical Extents (PE), typically 4MB each. Logical Volumes are allocated in units of PEs. This is why LV sizes are always a multiple of the PE size (usually 4MB).

```
# Check PE size
vgdisplay vg_data | grep 'PE Size'
# PE Size   4.00 MiB
```

Tip: The device path for a Logical Volume can be referenced in two ways: /dev/<VG>/<LV> (e.g., /dev/vg_data/lv_app) or /dev/mapper/<VG>-<LV> (e.g., /dev/mapper/vg_data-lv_app). Both refer to the same device — /etc/fstab and mount commands can use either form.

Q9. How do you create and mount an LVM volume?

Answer

The existing answer is correct and covers the core steps. This answer provides the complete process with all options, verification steps, and persistent mounting.

Complete Step-by-Step LVM Volume Creation

Step 1 — Prepare the Physical Volume

```
# Initialise /dev/sdb as a Physical Volume
pvcreate /dev/sdb
# Physical volume '/dev/sdb' successfully created.

# Multiple disks at once
pvcreate /dev/sdb /dev/sdc

# Verify
pvs
pvdisplay /dev/sdb
```

Step 2 — Create the Volume Group

```
# Create a VG named 'vg_data' using /dev/sdb
vgcreate vg_data /dev/sdb
# Volume group 'vg_data' successfully created

# Create VG with multiple disks
vgcreate vg_data /dev/sdb /dev/sdc

# Create VG with custom PE size (default 4MB)
vgcreate -s 8M vg_data /dev/sdb

# Verify
vgs
vgdisplay vg_data
```

Step 3 — Create the Logical Volume

```
# Create a 10GB LV named 'lv_data' in 'vg_data'
lvcreate -L 10G -n lv_data vg_data
# Logical volume 'lv_data' created.

# Create LV using percentage of free VG space
lvcreate -l 80%FREE -n lv_data vg_data    # 80% of free space
lvcreate -l 100%FREE -n lv_data vg_data   # All free space

# Verify
lvs
```

```
lvdisplay /dev/vg_data/lv_data
```

Step 4 — Create a Filesystem

```
# Format with ext4 (most common, supports online resize)
mkfs.ext4 /dev/vg_data/lv_data

# Format with XFS (good for large files, also supports online resize)
mkfs.xfs /dev/vg_data/lv_data

# Add a label for identification
mkfs.ext4 -L mydata /dev/vg_data/lv_data
```

Step 5 — Create Mount Point and Mount

```
# Create the directory
mkdir -p /data

# Mount (temporary — not persistent across reboots)
mount /dev/vg_data/lv_data /data

# Verify mount
df -h /data
mount | grep lv_data
```

Step 6 — Make it Persistent (fstab)

```
# Get the UUID of the filesystem
blkid /dev/vg_data/lv_data
# /dev/vg_data/lv_data: UUID='abc-123...' TYPE='ext4'

# Add to /etc/fstab
echo '/dev/vg_data/lv_data  /data  ext4  defaults  0 2' >> /etc/fstab

# Or using UUID (safer — survives device name changes)
echo 'UUID=abc-123...  /data  ext4  defaults  0 2' >> /etc/fstab

# Test fstab entry without rebooting
mount -a
df -h /data   # Verify it mounted

# Verify it will work after reboot
findmnt --verify
```

fstab Column Explanation

Column	Value
Device	/dev/vg_data/lv_data
Mount point	/data
Filesystem	ext4
Options	defaults
Dump	0
Pass	2

> *Tip: Always test your fstab entry with mount -a before rebooting. A bad fstab entry can cause the server to fail to boot. If this happens, you need to boot into recovery mode or single-user mode to fix it. Using UUID instead of /dev/vg_data/lv_data is safer because device names can change if disks are added or reordered.*

Q10. How do you extend a logical volume?

Answer

The existing answer is correct. Expanding with complete options, online vs offline extension, and filesystem-specific commands.

Prerequisites — Ensure VG Has Free Space

```
# Check available free space in the VG
vgs
# VG       VFree
```

```
# vg_data  50GB    ← 50GB available to extend into

# If VG is full, add a new disk first:
pvcreate /dev/sdc
vgextend vg_data /dev/sdc
```

Step 1 — Extend the Logical Volume

```
# Extend by a specific size
lvextend -L +5G /dev/vg_data/lv_data
# Size of logical volume vg_data/lv_data changed from 10.00 GiB to 15.00 GiB

# Extend to an exact size
lvextend -L 20G /dev/vg_data/lv_data

# Extend by percentage of free VG space
lvextend -l +100%FREE /dev/vg_data/lv_data
lvextend -l +50%FREE  /dev/vg_data/lv_data

# Extend AND resize filesystem in one command
lvextend -L +5G --resizefs /dev/vg_data/lv_data
# --resizefs detects filesystem type and runs appropriate resize
```

Step 2 — Grow the Filesystem (if not using --resizefs)

For ext4 Filesystem

```
# Resize ext4 filesystem to fill the LV (online — no unmount needed)
resize2fs /dev/vg_data/lv_data
# resize2fs 1.45.5 (07-Jan-2020)
# Filesystem at /dev/vg_data/lv_data is mounted on /data
# The filesystem is already 2621440 (4k) blocks long. Nothing to do!
# → After resize:
# The filesystem on /dev/vg_data/lv_data is now 3932160 (4k) blocks long.
```

For XFS Filesystem

```
# XFS: must use xfs_growfs with the MOUNT POINT (not device)
# XFS can ONLY grow (not shrink), and grows ONLINE
xfs_growfs /data
# or
xfs_growfs /dev/vg_data/lv_data
```

Verify the Extension

```
# Check LV size
lvs /dev/vg_data/lv_data

# Check filesystem size
df -h /data
# Filesystem              Size  Used Avail Use%
# /dev/mapper/vg_data-lv_data   15G  5.2G  8.8G  37%
```

Combined One-Liner (Recommended)

```
# Extend LV by 5GB AND resize the filesystem in one command
lvextend -L +5G --resizefs /dev/vg_data/lv_data
# Works with ext4, ext3, and XFS (auto-detected)
```

ext4 vs XFS Filesystem Commands

Operation	ext4
Grow filesystem	resize2fs /dev/<vg>/<lv>
Shrink filesystem	resize2fs /dev/<vg>/<lv> <size>
Online resize	Yes — no unmount needed
Unmount required	No for grow; Yes for shrink
Check filesystem	e2fsck -f /dev/<vg>/<lv>

> *Tip: Use lvextend -L +5G --resizefs as your default command. It handles both the LV extension and filesystem resize in one step, detects the filesystem type automatically, and works online without unmounting. This is the safest and most convenient approach.*

Q11. You are running out of space on /home. What would you do if it's an LVM volume?

Answer

The existing answer is correct and covers the steps. This answer provides the complete sequence with diagnostics, safety checks, and best practices.

Step 1 — Diagnose the Situation

```
# Confirm /home is on an LVM volume
df -h /home
mount | grep /home
# /dev/mapper/my_vg-home_lv on /home type ext4

# Get the VG and LV names
lvs | grep home
# LV      VG     Attr      LSize  Pool
# home_lv my_vg  -wi-ao--  50.00g

# Check VG free space
vgs my_vg
# VFree = 0   ← VG is full, need to add a disk
```

Step 2 — Add a New Physical Disk

```
# List available disks (find the new unpartitioned disk)
lsblk
fdisk -l | grep '^Disk /dev/'

# Identify the new disk (e.g., /dev/sdc)
# Verify it's the correct disk before proceeding
fdisk -l /dev/sdc

# Initialise as Physical Volume
pvcreate /dev/sdc
# Physical volume '/dev/sdc' successfully created.
```

Step 3 — Extend the Volume Group

```
# Add the new PV to the existing VG
vgextend my_vg /dev/sdc
# Volume group 'my_vg' successfully extended

# Verify VG now has free space
vgs my_vg
# VFree = 100GB   ← new space from /dev/sdc
```

Step 4 — Extend the Logical Volume and Filesystem

```
# Option A: Extend by a specific amount (e.g., add 10GB)
lvextend -L +10G --resizefs /dev/my_vg/home_lv

# Option B: Use all available free space in the VG
lvextend -l +100%FREE --resizefs /dev/my_vg/home_lv

# For XFS filesystem (common on RHEL/CentOS)
lvextend -L +10G /dev/my_vg/home_lv
xfs_growfs /home
```

Step 5 — Verify

```
df -h /home
# Filesystem                  Size  Used Avail Use%
# /dev/mapper/my_vg-home_lv   60G   48G   10G  83%

lvs my_vg
# home_lv  60.00g   ← extended from 50GB to 60GB
```

Alternative: If VG Still Has Free Space (No New Disk Needed)

```
# Check VG free space first
vgs my_vg
# If VFree > 0, skip pvcreate/vgextend steps

# Go directly to step 4
lvextend -L +10G --resizefs /dev/my_vg/home_lv
```

Alternative: Reclaim Space Before Extending

```
# Find large files in /home
find /home -type f -size +500M -exec ls -lh {} \;
```

```
du -sh /home/* | sort -rh | head -20

# Find old files not accessed in 180 days
find /home -type f -atime +180 -exec ls -lh {} \;

# Check for large hidden files
find /home -name '.*' -size +100M
```

> *Tip: Before extending, always identify what filled the disk. If a user or process is generating data at an unusual rate, extending buys time but doesn't fix the root cause. Use find and du to identify the largest directories and files, then investigate why they grew.*

Q12. Can you reduce the size of an LVM volume?

Answer

The existing answer is correct. Shrinking an LVM volume is significantly more dangerous than extending — it requires unmounting, filesystem shrinking (filesystem first, LV second), and careful verification at each step. A mistake can cause data loss.

Important Warning — Shrink Order is Critical

> *Note: NEVER shrink the LV before shrinking the filesystem. If the LV is made smaller than the filesystem, the filesystem metadata at the end of the volume is destroyed — resulting in permanent, unrecoverable data corruption. ALWAYS: shrink filesystem first, then shrink LV.*

Complete Safe Shrink Process

Step 1 — Check Current Usage

```
# You cannot shrink below the used space
df -h /data
# Size=20G  Used=4.2G  Avail=14G  → can safely shrink to ~8GB (with headroom)

lvs /dev/vg_data/lv_data
# LSize = 20.00g
```

Step 2 — Unmount the Filesystem

```
# Shrinking REQUIRES the filesystem to be unmounted
umount /data

# If umount fails (busy), find what's using it
lsof +D /data
fuser -mv /data
# Then kill or stop those processes
```

Step 3 — Run Filesystem Check

```
# e2fsck MUST be run before resizing ext4
e2fsck -f /dev/vg_data/lv_data
# -f = force check even if filesystem seems clean
# Fix any errors before proceeding

# For XFS: XFS CANNOT be shrunk — this limitation is by design
# If using XFS, you must backup, delete, and recreate
```

Step 4 — Shrink the Filesystem FIRST

```
# Shrink ext4 filesystem to 5GB
resize2fs /dev/vg_data/lv_data 5G
# resize2fs: Filesystem has been shrunk from ...
# resize2fs: The filesystem on /dev/vg_data/lv_data is now ...

# Important: resize2fs 5G means filesystem becomes 5GB
# Always leave headroom — don't shrink to exactly the used space
# If data is 4GB, shrink to 6GB minimum for safety
```

Step 5 — Shrink the Logical Volume SECOND

```
# Now shrink the LV to match the new filesystem size
lvreduce -L 5G /dev/vg_data/lv_data
# WARNING: Reducing active logical volume to 5.00 GiB.
# THIS MAY DESTROY YOUR DATA (filesystem etc.)
# Do you really want to reduce vg_data/lv_data? [y/n]: y
# Logical volume vg_data/lv_data successfully resized.
```

```
# Alternative: resize both in one command (careful!)
# lvreduce --resizefs -L 5G /dev/vg_data/lv_data
# Only use --resizefs if you're confident about the target size
```

Step 6 — Run Filesystem Check Again

```
# Verify filesystem is intact after shrinking
e2fsck -f /dev/vg_data/lv_data
# Should show: clean, 1234/... files, .../... blocks
```

Step 7 — Remount and Verify

```
mount /dev/vg_data/lv_data /data
df -h /data
# Now shows: 5GB total

lvs /dev/vg_data/lv_data
# LSize = 5.00g
```

XFS Cannot Be Shrunk — Alternative Approach

```
# For XFS volumes, you must take a different approach:
# 1. Backup all data
rsync -av /data/ /backup/data/

# 2. Unmount and destroy the LV
umount /data
lvremove /dev/vg_data/lv_data

# 3. Recreate with new smaller size
lvcreate -L 5G -n lv_data vg_data
mkfs.xfs /dev/vg_data/lv_data
mount /dev/vg_data/lv_data /data

# 4. Restore data
rsync -av /backup/data/ /data/
```

> *Tip: Always take a snapshot or full backup before shrinking an LVM volume. Even with perfect execution of all steps, shrinking is riskier than extending. Test the process in a non-production environment first. For production systems, the backup/restore approach using XFS is often safer than the online shrink approach.*

Q13. How do LVM snapshots work?

Answer

The existing answer is correct and covers the basics. Expanding with complete implementation details, use cases, and best practices.

How LVM Snapshots Work Internally

An LVM snapshot uses a Copy-on-Write (CoW) mechanism. When you take a snapshot:

- A new Logical Volume is created — the snapshot LV. Initially it's nearly empty.
- The snapshot LV stores only a map of what the origin LV looked like at snapshot time.
- When the origin LV's data changes, the original data is copied to the snapshot LV BEFORE the change is written.
- The snapshot always represents the state at creation time — regardless of how much the origin changes afterward.
- Reading from the snapshot shows the original data (from CoW copies + unchanged original blocks).

```
Origin LV (before snapshot):  [A] [B] [C] [D]
                                    ↓ take snapshot
Snapshot LV created (empty):  [ ] [ ] [ ] [ ]
                                    ↓ B is modified in origin
Before writing new B:  old B is copied to snapshot
Snapshot LV now has:   [ ] [B] [ ] [ ]   ← original B preserved
Origin LV now has:     [A] [B'] [C] [D]  ← B' is the new value
Reading snapshot:      [A] [B]  [C] [D]  ← still sees original data
```

Creating a Snapshot

```
# Create a snapshot of lv_data
# -s = snapshot, -L = snapshot volume size, -n = name
lvcreate -s -L 5G -n lv_data_snap /dev/vg_data/lv_data
# Logical volume 'lv_data_snap' created.

# The snapshot size (5G) must be large enough to hold
# all changes made to the origin after snapshot creation.
```

```
# If the snapshot fills up, it becomes invalid (corrupted).

# Check snapshot status
lvs -a vg_data
# LV           Attr       LSize  Pool Origin    Snap%
# lv_data      owi-aos--- 50.00g
# lv_data_snap swi-a-s---  5.00g      lv_data   12.50
# Snap% = 12.5% of snapshot space used
```

Use Case 1 — Pre-Upgrade Backup

```
# Before a major application upgrade:
lvcreate -s -L 10G -n lv_app_pre_upgrade /dev/vg_data/lv_app
# Perform the upgrade
./upgrade.sh

# If upgrade fails — INSTANT rollback:
# Stop the application
systemctl stop myapp
umount /opt/app

# Merge snapshot back into origin
lvconvert --merge /dev/vg_data/lv_app_pre_upgrade
# After reboot, the volume is restored to snapshot state

# Or manually restore:
dd if=/dev/vg_data/lv_app_pre_upgrade of=/dev/vg_data/lv_app
```

Use Case 2 — Consistent Backup

```
# Mount the snapshot read-only for backup
mkdir -p /mnt/snapshot
mount -o ro /dev/vg_data/lv_data_snap /mnt/snapshot

# Backup from the consistent snapshot
rsync -av /mnt/snapshot/ /backup/data/
tar -czf /backup/data_$(date +%Y%m%d).tar.gz -C /mnt/snapshot .

# Unmount and remove snapshot when done
umount /mnt/snapshot
lvremove -f /dev/vg_data/lv_data_snap
```

Monitoring Snapshot Usage

```
# Watch snapshot fill percentage
watch -n 5 "lvs -a | grep snap"

# Alert if snapshot > 70% full
snap_pct=$(lvs --noheadings -o snap_percent /dev/vg_data/lv_data_snap | tr -d ' ')
if (( $(echo "$snap_pct > 70" | bc -l) )); then
    echo "WARNING: Snapshot ${snap_pct}% full"
fi
```

> *Tip: Size the snapshot at least 20% of the origin LV for short-lived snapshots (hours). For backups that may take longer, size it larger. A snapshot that fills up becomes INVALID and cannot be used — you lose your safety net silently. Monitor snap_percent in Prometheus or a cron job.*

Q14. What's the difference between striped and linear LVM volumes?

Answer

The existing answer is correct. Expanding with detailed implementation, performance characteristics, and real-world use cases.

Linear Logical Volumes — Default Behaviour

In a linear LV, data is written sequentially across Physical Volumes. The first PV is filled before the second PV is used. If the LV spans two disks, all I/O goes to disk 1 until it's full, then switches to disk 2. No performance parallelism.

```
# Create a linear LV spanning two PVs
lvcreate -L 100G -n lv_linear vg_data
# Data layout:
# Disk 1 (sdb): [0GB ... 80GB] ← filled first
# Disk 2 (sdc): [80GB ... 100GB] ← used after disk 1 full
```

```
# All reads/writes go to one disk at a time
```

Striped Logical Volumes — RAID-0 Style

In a striped LV, data is split into chunks (stripes) and written across multiple PVs simultaneously. A 64KB stripe size means each 64KB chunk is written to a different disk in round-robin. This enables parallel I/O across multiple disks.

```
# Create a striped LV across 2 PVs with 64KB stripe size
lvcreate -i 2 -I 64 -L 100G -n lv_striped vg_data
# -i 2 = 2 stripes (2 disks)
# -I 64 = 64KB stripe size
# -L 100G = 50GB used on each disk (total 100GB)

# Create striped LV across 3 PVs
lvcreate -i 3 -I 64 -L 300G -n lv_fast_db vg_data /dev/sdb /dev/sdc /dev/sdd

# Data layout:
# Disk 1 (sdb): [stripe 0] [stripe 3] [stripe 6] ...
# Disk 2 (sdc): [stripe 1] [stripe 4] [stripe 7] ...
# Disk 3 (sdd): [stripe 2] [stripe 5] [stripe 8] ...
# All 3 disks write simultaneously!
```

Performance Comparison

Characteristic	Linear
Sequential read/write	Limited to one disk speed
Random I/O	Limited to one disk IOPS
Disk failure impact	Lose only the failed disk's data
Complexity	Simple — default behaviour
Best for	General purpose storage
Reliability	Higher — partial loss possible
Typical stripe size	N/A

Verify Stripe Configuration

```
lvdisplay /dev/vg_data/lv_striped
# LV Status: available
# Stripes:    2
# Stripe size: 64.00 KiB

# See full segment details
lvs -a -o +stripes,stripe_size,seg_type
```

Choosing Stripe Size

Workload	Recommended Stripe Size
Database (random I/O)	64KB - 256KB
Video streaming (sequential)	256KB - 1MB
Mixed workload	128KB
Log files (sequential write)	512KB - 1MB

> *Note: Striped LVs have no redundancy — a single disk failure destroys ALL data on the striped volume. For production databases requiring both performance AND reliability, combine LVM striping with RAID-1 (mirroring) at the hardware or MD RAID level, or use RAID-10 which stripes across mirrored pairs.*

Q15. What is thin provisioning in LVM?

Answer

The existing answer is correct. Expanding with implementation details, monitoring, and real-world scenarios.

What is Thin Provisioning

Traditional LVM allocates physical storage immediately when you create a logical volume — a 50GB LV immediately consumes 50GB of VG space, even if the LV is empty. Thin provisioning changes this: the LV appears to be 50GB but physical storage is only allocated as data is actually written.

Architecture — Thin Pool and Thin Volumes

```
VG (vg_data, 100GB physical)
  └── Thin Pool (pool_data, 100GB allocated from VG)
       ├── Thin LV A (appears 50GB, uses 10GB physical)
       ├── Thin LV B (appears 50GB, uses 15GB physical)
       └── Thin LV C (appears 50GB, uses 5GB physical)

Total apparent: 150GB  ←  over-committed (more than 100GB)
Total physical used: 30GB  ←  actual storage consumed
```

Creating a Thin Pool and Thin Volumes

```
# Step 1: Create a Thin Pool LV
lvcreate -L 100G --thinpool pool_data vg_data
# Logical volume 'pool_data' created.
# WARNING: Sum of all thin volume sizes exceeds the size of thin pool.

# Step 2: Create Thin Volumes (virtual volumes from the pool)
lvcreate -V 50G --thin -n lv_app  vg_data/pool_data
lvcreate -V 50G --thin -n lv_db   vg_data/pool_data
lvcreate -V 50G --thin -n lv_logs vg_data/pool_data
# Total provisioned: 150GB from a 100GB pool — overcommitted

# Format and use like a normal LV
mkfs.ext4 /dev/vg_data/lv_app
mount /dev/vg_data/lv_app /opt/app
```

Monitoring Thin Pool Usage

```
# Check pool usage
lvs -a vg_data
# LV         Attr        LSize  Pool      Data%  Meta%
# pool_data  twi-aotz--  100GB             32.5%  1.2%
# lv_app     Vwi-aotz--   50GB pool_data
# lv_db      Vwi-aotz--   50GB pool_data

# Data% = 32.5% of 100GB used = 32.5GB physically allocated

# Alert when pool reaches threshold
lvs --noheadings -o data_percent vg_data/pool_data | \
  awk '{if ($1 > 80) print "WARNING: Thin pool at " $1 "%"}'
```

Thin Snapshots

Thin volumes enable highly efficient snapshots. Because both the origin and snapshot can share thin-allocated blocks, snapshots of thin volumes consume virtually no additional space initially:

```
# Create a thin snapshot (very efficient)
lvcreate -s --name lv_app_snap /dev/vg_data/lv_app
# No size required — uses thin pool space dynamically
```

When to Use Thin Provisioning

Use Case	Benefit
Development/test VMs	Provision 10 VMs with 100GB each from a 500GB pool
Container storage	Many containers share a pool; most use very little
Databases with varying growth	Allocate large LV without wasting space early on
Pre-production environments	Look identical to prod without identical storage cost
Many small services	Right-size apparent allocation without fragmentation

> *Note: Thin provisioning requires careful monitoring. If the thin pool runs out of physical space while volumes are still being written to, writes fail and the thin volumes can become corrupted. Always alert when the pool reaches 70-80% and have a plan to expand the pool (add more PVs to the VG and extend the pool LV).*

Q16. How do you back up and restore LVM configuration?

Answer

The existing answer is correct and brief. Expanding with complete details, automated backup, and disaster recovery procedures.

How LVM Configuration Backup Works

LVM automatically maintains configuration backups in two locations on every VG change (vgcreate, lvcreate, vgextend, etc.):

- /etc/lvm/backup/ — Contains the most recent backup of each VG configuration. One file per VG, overwritten on each change.
- /etc/lvm/archive/ — Contains historical backups with timestamps. Every configuration change creates a new archived file. Default: 10 archives kept per VG.

```
# View current LVM backup
cat /etc/lvm/backup/vg_data

# List archived configurations
ls -lt /etc/lvm/archive/ | head -20
# vg_data_00001-123456789.vg
# vg_data_00002-123456790.vg
# (sorted newest first)

# View a specific archive
cat /etc/lvm/archive/vg_data_00001-123456789.vg
```

Manual Backup — Exporting VG Metadata

```
# Export VG configuration to a specific file
vgcfgbackup -f /backup/lvm/vg_data_$(date +%Y%m%d).cfg vg_data

# Backup ALL volume groups
vgcfgbackup
# Writes to /etc/lvm/backup/ for each VG

# Include LVM backup in general system backup
tar -czf /backup/lvm_config_$(date +%Y%m%d).tar.gz \
  /etc/lvm/backup/ /etc/lvm/archive/
```

Automated LVM Backup Script

```
#!/bin/bash
# /etc/cron.daily/lvm-backup
BACKUP_DIR='/backup/lvm'
DATE=$(date +%Y%m%d)
mkdir -p "$BACKUP_DIR"

# Backup all VG configurations
vgcfgbackup

# Archive to remote location
tar -czf "$BACKUP_DIR/lvm_${DATE}.tar.gz" /etc/lvm/

# Keep only 30 days of backups
find "$BACKUP_DIR" -name 'lvm_*.tar.gz' -mtime +30 -delete
echo "LVM backup completed: $BACKUP_DIR/lvm_${DATE}.tar.gz"
```

Restoring LVM Configuration

Scenario 1 — Restore After Accidental VG Modification

```
# List available archives to find the correct restore point
ls -lt /etc/lvm/archive/

# Restore from the most recent archive
vgcfgrestore vg_data

# Restore from a specific archive file
vgcfgrestore -f /etc/lvm/archive/vg_data_00005-123456790.vg vg_data

# Restore from a manual backup
vgcfgrestore -f /backup/lvm/vg_data_20240115.cfg vg_data
```

Scenario 2 — Disaster Recovery (New Server)

```
# After replacing failed disks or moving to new server:

# 1. Install the disks and boot the system
```

```
# 2. Scan for Physical Volumes
pvscan

# 3. If VG is found, activate it
vgchange -ay vg_data

# 4. If VG metadata is corrupted, restore from backup
vgcfgrestore -f /backup/lvm/vg_data_20240115.cfg vg_data

# 5. Activate and mount
vgchange -ay vg_data
mount /dev/vg_data/lv_data /data
```

Additional System-Level Backup Tools

```
# pvs/vgs/lvs — export full configuration as text
pvs > /backup/lvm/pvs_$(date +%Y%m%d).txt
vgs > /backup/lvm/vgs_$(date +%Y%m%d).txt
lvs > /backup/lvm/lvs_$(date +%Y%m%d).txt

# Full LVM report
lvm fullreport > /backup/lvm/fullreport_$(date +%Y%m%d).txt

# dmsetup — backup device mapper table (includes LVM)
dmsetup table > /backup/lvm/dmsetup_$(date +%Y%m%d).txt
```

LVM Backup Coverage

What Is Backed Up	What Is NOT Backed Up
VG structure (PV, VG, LV names and sizes)	Actual data on the LVs
LV attributes (striping, snapshots, thin pools)	Filesystem metadata (ext4, xfs journal)
UUID and device names	Application data
PE/LE allocation maps	Databases, files

> *Note: LVM configuration backup only saves the LVM metadata — not the actual data. A vgcfgrestore restores the LVM structure so you can access the volumes, but the data must still be intact on the physical disks. Always combine LVM metadata backup with filesystem-level data backup (rsync, tar, Velero, etc.).*
